

- Calcul Distribué : Hadoop -

TP 1 : Manipulation de l'HDFS via CLI & API Java

⇒ Objectifs du TP

Apprendre à manipuler l'HDFS via :

1. CLI : Commande Ligne Interface

- Manipuler le système de fichier HDFS en ligne de commandes,
- Appliquer les commandes vues en cours et savoir se documenter sur d'autres,
- Différencier entre la manipulation des fichiers/dossiers locaux et HDFS.

2. API Java

- Apprendre les étapes de configuration, création et exécution d'un programme HDFS en Java.

Partie I : HDFS en ligne de commandes

1. Lancez la commande "**hdfs dfs**". Regardez la liste de commandes proposées par hdfs. Vous pouvez obtenir de l'aide sur chaque commande en tapant:
/hdfs dfs -usage VOTRECOMMANDE
2. Afficher le contenu de la racine HDFS (vous pouvez descendre inspecter les dossiers que vous voyez).
⇒ **Remarque** : Il n'y a pas de commande équivalente à *cd*, parce qu'il n'y a pas de notion de dossier courant dans l'HDFS, donc à chaque fois, il faut remettre le chemin complet.
3. Créer un répertoire à votre nom dans la racine HDFS (Pensez à précéder votre commande avec **sudo -u hdfs** pour avoir la permission de manipulation en HDFS). Copier deux fichiers txt locaux de votre choix dans ce répertoire. Que remarquez-vous ?
4. Modifier les droits d'accès à ce répertoire pour résoudre le problème précédent. Recopier les fichiers en HDFS et vérifier la réussite de cette opération.
5. Transfère l'un des fichiers précédents de HDFS vers le local en lui changeant son nom (Cette commande ne serait pas à faire avec de vraies méga-données).

6. Concaténer le contenues des fichiers précédents et stocker le résultat dans un fichier local.
⇒ **Indication** : se documenter sur la commande: *getmerge*
7. Supprimer le répertoire (avec son contenu) crée en HDFS et les fichiers locaux manipulés précédemment.

Partie II : HDFS via l'API Java

Hadoop propose une API Java complète pour accéder aux fichiers de HDFS. Elle repose sur deux classes principales :

- **FileSystem** : représente l'arbre des fichiers (*file system*). Cette classe permet de copier des fichiers locaux vers HDFS (et inversement), renommer, créer et supprimer des fichiers et des dossiers
- **FileStatus** : gère les informations d'un fichier ou dossier : taille avec *getLen()* et nature avec *isDirectory()* ou *isFile()*.

Ces deux classes ont besoin de connaître la configuration du cluster HDFS, à l'aide de la classe **Configuration**. D'autre part, les noms complets des fichiers sont représentés par la classe **Path**.

Exemple 1 : Lecture et Affichage d'un fichier HDFS

```
public class HDFSread {  
    public static void main(String[] args) throws IOException {  
        Configuration conf = new Configuration();  
        FileSystem fs = FileSystem.get(conf);  
        Path nomcomplet = new Path("/dataSetTP/dataWC.txt");  
        FSDataInputStream inStream = fs.open(nomcomplet);  
        InputStreamReader isr = new InputStreamReader(inStream);  
        BufferedReader br = new BufferedReader(isr);  
        String line;  
        while ((line = br.readLine()) != null){  
            System.out.println (line);  
        }  
        inStream.close();  
        fs.close();  
    }  
}
```

Exemple 2 : Création d'un fichier en HDFS

```
public class HDFSwrite {  
    public static void main(String[] args) throws IOException {  
        Configuration conf = new Configuration();  
        FileSystem fs = FileSystem.get(conf);  
        Path nomcomplet = new Path("/dataSetTP/testHdfsWrite.txt");  
  
        if (! fs.exists(nomcomplet)) {  
            FSDataOutputStream outputStream = fs.create(nomcomplet);  
            outputStream.writeBytes("Bonjour tout le monde !");  
            outputStream.close();  
        }  
        fs.close();  
    }  
}
```

Pour exécuter un tel programme, on procède comme suit :

- **Etape 1 :** Ecrire le code source sur votre IDE (Eclipse par ex) en important les jars Hadoop nécessaires.
- **Etape 2 :** Exporter la source sous format jar.
- **Etape 3 :** Exécuter le programme sur hadoop via la commande suivante :

```
hadoop jar <jar_name>.jar <package_name>/main_class_name [input_param]
```

Après avoir tester les programmes précédents, créer et exécuter des programmes JAVA pour les différents cas ci-dessous:

- **Cas 1 :**
Modifier le premier exemple pour compter et afficher le nombre de lignes d'un fichier spécifié lors de l'exécution du programme.
- **Cas 2 :**
Ecrire un programme qui permet d'ouvrir un fichier local et copier son contenu dans un fichier HDFS crée automatiquement.
- **Cas 3 :**
Ecrire un programme qui concatène un ensemble de fichiers locaux dans un fichier HDFS.